

FUNDAMENTALS OF DATABASE MANAGEMENT SYSTEMS

Academic Reference Manual · Module I: Architecture, Relational Model, and Keys

1. INTRODUCTION TO DATABASE SYSTEMS

A Database Management System (DBMS) is specialized system software designed to facilitate the creation, maintenance, querying, and administrative control of structured collections of data. Prior to the adoption of DBMS technology, data management relied heavily on conventional file processing systems. These legacy setups exhibited severe structural shortcomings, including pronounced data redundancy, program-data interdependence, access anomalies, concurrent access limitations, and inadequate integrity enforcement.

2. THE ANSI-SPARC THREE-SCHEMA ARCHITECTURE

To decouple user applications from physical storage concerns, the ANSI-SPARC framework defines three distinct levels of abstraction, providing systematic physical and logical data independence:

- **External Level (View Schema):** The highest level of data abstraction, addressing tailored user views. Individual users interact with specialized interfaces and access only the subset of data relevant to their role, hiding extraneous details and underlying schemas.
- **Conceptual Level (Logical Schema):** Defines the structural blueprint of the entire database entity ecosystem, specifying logical entities, attributes, relationships, security constraints, and semantic business rules. It remains independent of physical storage implementations.
- **Internal Level (Physical Schema):** The low-level interface that specifies how records are physically organized, indexed, clustered, and allocated across storage volumes and underlying blocks.

Data Independence Principle: *Logical Data Independence* allows changes to the conceptual schema without altering external views or existing application code. *Physical Data Independence* guarantees that internal storage modifications (such as changing index types or reorganizing files across disk blocks) require no alterations to conceptual or external schemas.

3. RELATIONAL MODEL FOUNDATIONS

Formulated by Edgar F. Codd, the relational model structures information exclusively as mathematical relations, represented as two-dimensional tables consisting of rows and columns:

- **Tuple:** A single row in a relation representing an individual entity instance or relationship record.
- **Attribute:** A named column describing a specific property or feature of the relation's domain.
- **Domain:** The strict set of permissible atomic values acceptable for an attribute.
- **Cardinality and Degree:** Cardinality designates the total number of tuples in a relation, whereas Degree corresponds to the total count of attributes.

4. KEY CONSTRAINTS AND CLASSIFICATIONS

Keys ensure entity identification, prevent ambiguous records, and uphold relational linkages across the system:

- **Super Key:** Any attribute or minimal/non-minimal combination of attributes that uniquely identifies an individual tuple within a relation.
- **Candidate Key:** A minimal super key containing no extraneous attributes; possesses the property of uniqueness and irreducible minimality.
- **Primary Key:** The specific candidate key chosen by the database architect as the unique identifier for the relation. It enforces absolute uniqueness and cannot contain null values (Entity Integrity Rule).

5. RELATIONAL ALGEBRA FUNDAMENTALS

Relational algebra serves as the procedural query framework providing the formal foundation for relational database operations. Operations take one or more relations as input and produce a new relation:

- **Selection (σ):** Unary operator selecting horizontal subsets of tuples satisfying a given predicate condition ($\sigma_{\text{Predicate}}(R)$).
- **Projection (π):** Unary operator selecting vertical subsets of attributes while eliminating resulting duplicate rows ($\pi_{\text{Attributes}}(R)$).
- **Cartesian Product ($\times$):** Binary operator pairing every tuple of relation R with every tuple of relation S.
- **Natural Join ($\bowtie$):** Enforces equality over attributes possessing identical names in both relations, projecting unique schema attributes.
- **Set Operations:** Union ($\cup$), Intersection ($\cap$), and Set Difference ($-$), all requiring operand relations to satisfy strict union-compatibility.

6. DATABASE NORMALIZATION AND NORMAL FORMS

Normalization is the systematic process of decomposing anomalous relations to mitigate insertion, deletion, and update anomalies while minimizing non-essential redundancy through functional dependency analysis.

The first stage, First Normal Form (1NF), mandates that every attribute within a relation must contain solely atomic, indivisible values, explicitly disallowing multi-valued attributes, repeating groups, and nested composite structures. By flattening multi-valued elements into separate rows or relations, 1NF establishes a uniform tabular structure where each row-column intersection yields a single discrete datum.

Second Normal Form (2NF) advances relational hygiene by requiring full functional dependency of all non-key attributes on candidate keys. A relation residing in 1NF achieves 2NF status only when no non-prime attribute depends on a proper subset of any candidate key. This rule directly resolves partial dependencies that occur when relations utilize composite primary keys, thereby preventing update inconsistencies whenever shared composite attributes are altered.

Third Normal Form (3NF) further refines the schema by mandating that a relation in 2NF must contain no transitive dependencies among its non-prime attributes. Specifically, for every non-trivial functional dependency $X \rightarrow Y$, either X must be a superkey or Y must be a prime attribute. This eliminates scenarios where non-key attributes determine other non-key attributes, ensuring that changes to independent entities do not propagate unintentionally throughout unrelated columns.

Boyce-Codd Normal Form (BCNF) serves as a stricter, more comprehensive extension of 3NF. Under BCNF, for every non-trivial functional dependency $X \rightarrow Y$, the determinant X must strictly be a superkey, regardless of whether Y is a prime or non-prime attribute. This stringent standard eliminates lingering anomalies caused by overlapping candidate keys with multiple attributes, guaranteeing that every determinant in the schema functions as a unique row identifier.

7. STRUCTURED QUERY LANGUAGE (SQL) TAXONOMY

SQL is the standardized declarative language for manipulating relational databases. Its instructions fall into four core operational categories:

- **Data Definition Language (DDL):** Defines, modifies, and removes database structures (CREATE, ALTER, DROP, TRUNCATE).
- **Data Manipulation Language (DML):** Queries and manages records within established structures (SELECT, INSERT, UPDATE, DELETE).
- **Data Control Language (DCL):** Regulates user privileges and system access rights (GRANT, REVOKE).
- **Transaction Control Language (TCL):** Governs execution boundaries and state transitions (COMMIT, ROLLBACK, SAVEPOINT).

8. TRANSACTION PROCESSING AND ACID PROPERTIES

A transaction is a logical unit of database processing that includes one or more access operations. To preserve data consistency during concurrent execution and system failures, transactions must satisfy the ACID criteria:

- **Atomicity:** The all-or-nothing principle. A transaction must either run entirely to completion or undergo total reversal via rollback operations in case of failure. Managed by the recovery manager.
- **Consistency:** Execution of a transaction in isolation must preserve structural and semantic database invariants without violating any defined integrity constraints.
- **Isolation:** Concurrently executing transactions must remain unaware of each other's intermediate states. Controlled directly by the concurrency control subsystem.
- **Durability:** Once a transaction successfully commits, its state modifications persist reliably, remaining immune to system crashes and power failures. Managed by log write-ahead mechanisms.

9. CONCURRENCY CONTROL AND SERIALIZABILITY

Unregulated concurrent access causes common execution anomalies, including Dirty Reads, Lost Updates, Non-Repeatable Reads, and Phantom Reads.

- **Serializability:** The gold standard of correctness. A concurrent execution schedule is serializable if its outcome is equivalent to that of some purely serial execution of the same transactions.
- **Two-Phase Locking (2PL):** A protocol guaranteeing conflict serializability by enforcing two distinct operational phases:
 1. *Growing Phase:* Transactions may acquire new locks (shared or exclusive) but cannot release existing locks.
 2. *Shrinking Phase:* Transactions may release acquired locks but cannot request any new locks.
- **Deadlock Conditions:** Strict locking protocols may lead to deadlocks, requiring prevention policies, timeout mechanisms, or detection via Wait-For Graphs (WFG).

10. PHYSICAL STORAGE ORGANIZATION AND INDEXING

Physical storage structures dictate disk input/output efficiency, data retrieval speeds, and storage consumption:

- **Primary and Clustered Indexes:** An index defined over an ordered data file where the search key determines the physical sequential order of data records on storage blocks. A table can possess only one clustered index.
- **Secondary (Non-Clustered) Indexes:** An auxiliary structure built over fields that do not dictate physical data ordering. Contains pointers back to corresponding primary index rows or raw row identifiers (RIDs).
- **B+ Tree Index Structures:** The ubiquitous balanced search tree structure utilized in commercial DBMS engines. Leaf nodes are sequentially linked via doubly linked lists to facilitate range scans, while non-leaf nodes store search keys purely for directing search navigation paths. All leaves remain at identical tree depth, providing guaranteed $O(\log N)$ lookup, insertion, and deletion complexity.